CO2-AT1 PART-1

INDUSTRY PROBLEM

VANILLA POLICY GRADIENT VS REINFORCE

Name: K. SRI LASYA
Reg. No: 192472216
Course Code: MLA0305
Course Name: REINFORCEMENT LEARNING

AIM

To implement and compare Vanilla Policy Gradient and REINFORCE reinforcement learning methods for solving the CartPole-v1 control problem and evaluate their learning performance using episode rewards.

INDUSTRY PROBLEM

Reinforcement learning is widely used in robotics, autonomous systems, gaming, and control applications where an agent must learn suitable actions through interaction with an environment.

In many real-world control problems, an agent needs to learn a policy that maximizes the cumulative reward over time. Policy-gradient methods are useful because they directly optimize the policy instead of learning only a value function.

Two policy-gradient approaches are compared in this experiment:

- **Vanilla Policy Gradient (VPG):** Uses discounted returns as learning signals and normalizes them to improve training stability.
- **REINFORCE:** Uses the complete Monte Carlo return from each episode directly to update the policy.

The comparison focuses on learning performance, final average reward, best episode reward, and convergence behavior.

OBJECTIVES

1. Implement a policy neural network for action selection.
2. Train an agent using the Vanilla Policy Gradient method.
3. Train an agent using the REINFORCE method.
4. Calculate discounted returns for each episode.
5. Compare the learning performance of both algorithms.
6. Analyze the final average reward and best episode reward.
7. Visualize and interpret the learning curves.

DATASET / ENVIRONMENT DESCRIPTION

The experiment uses the **CartPole-v1** environment from Gymnasium instead of a conventional static dataset.

The major environment characteristics are:

- **Environment:** CartPole-v1
- **State size:** 4
- **Action size:** 2
- **Training episodes:** 300
- **Discount factor (γ):** 0.99
- **Learning rate:** 0.001
- **Optimizer:** Adam
- **Hidden layer:** 128 neurons
- **Activation function:** ReLU
- **Output activation:** Softmax

The four state variables describe the position and velocity of the cart and the angle and angular velocity of the pole.

METHODOLOGY

Step 1 – Environment Setup

The Gymnasium CartPole-v1 environment is initialized. PyTorch is used to construct and train the policy network, while NumPy, Pandas, and Matplotlib are used for numerical processing, result comparison, and visualization.

A fixed random seed is used to improve reproducibility of the experiment.

Step 2 – Policy Network

A neural-network-based policy is created with:

- Input layer containing 4 state values.
- Hidden layer containing 128 neurons.
- ReLU activation.
- Output layer containing 2 action probabilities.
- Softmax activation for converting network outputs into action probabilities.

The policy network can be represented as:

State → Linear Layer → ReLU → Linear Layer → Softmax → Action Probability

The agent samples an action from the probability distribution generated by the policy.

Step 3 – Discounted Return Calculation

For every episode, the cumulative discounted return is calculated from the rewards received by the agent.

The return is calculated as:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots \qquad \text{Eq(1)}$$

where:

- G_t = discounted return at time t
- R_t = reward at time t
- γ = discount factor
- **t** = current time step

From Eq(1), A discount factor of **0.99** is used in this experiment.

Step 4 – Vanilla Policy Gradient

The Vanilla Policy Gradient method uses the calculated returns as the learning signal.

For stable learning, the returns are normalized using their mean and standard deviation.

The loss is calculated using:

$$\text{Loss} = -\Sigma \log \pi(a_t|s_t)\, A_t$$

where:

- $\pi(a_t|s_t)$ = probability of selecting action a_t in state s_t
- A_t = normalized return used as the advantage signal.

The Adam optimizer updates the policy-network parameters after every episode.

Step 5 – REINFORCE

The REINFORCE algorithm uses the complete Monte Carlo return obtained from the episode.

The policy loss is calculated as:

$$\text{Loss} = -\Sigma \log \pi(a_t|s_t)\, G_t$$

where:

- $\pi(a_t|s_t)$ = probability of the selected action
- G_t = discounted return from the current time step.

Unlike the VPG implementation, the REINFORCE implementation directly uses the Monte Carlo returns without normalization.

Step 6 – Training

Both algorithms are trained separately for **300 episodes**.

For each episode:

1. The environment is reset.
2. The current state is passed to the policy network.
3. An action is sampled from the generated probability distribution.
4. The action is performed in the environment.

5. The reward and next state are recorded.
6. The episode continues until termination.
7. Discounted returns are calculated.
8. The policy loss is computed.
9. The network parameters are updated.

The average reward for every 50 episodes is displayed to observe the learning progress.

Step 7 – Performance Comparison

The final performance is compared using:

- Final 50-episode average reward
- Best episode reward
- Learning curve
- Overall training behavior

The obtained results are:

Algorithm	Final 50 Episode Average	Best Episode Reward
Vanilla Policy Gradient	12.98	48.0
REINFORCE	57.18	188.0

The final 50-episode average reward of REINFORCE is substantially higher than that of Vanilla Policy Gradient.

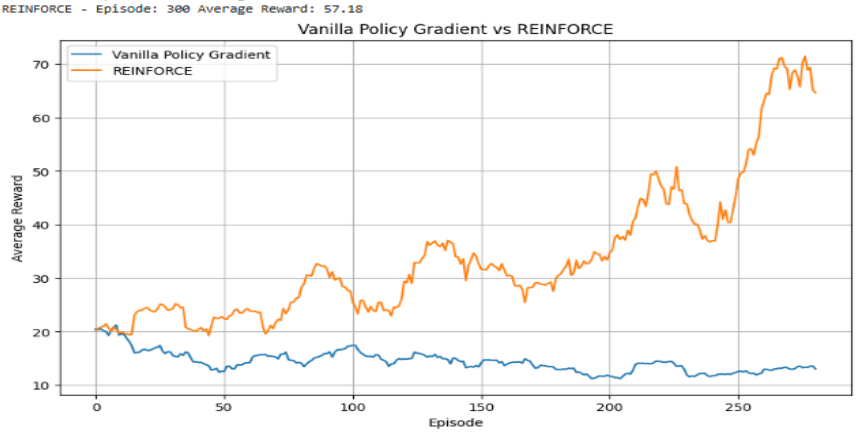
Step 8 – Visualization

```
Training Vanilla Policy Gradient...

VPG - Episode: 50 Average Reward: 17.86
VPG - Episode: 100 Average Reward: 13.94
VPG - Episode: 150 Average Reward: 15.96
VPG - Episode: 200 Average Reward: 13.7
VPG - Episode: 250 Average Reward: 12.54
VPG - Episode: 300 Average Reward: 12.98

Training REINFORCE...

REINFORCE - Episode: 50 Average Reward: 22.08
REINFORCE - Episode: 100 Average Reward: 24.8
REINFORCE - Episode: 150 Average Reward: 30.16
REINFORCE - Episode: 200 Average Reward: 29.78
REINFORCE - Episode: 250 Average Reward: 41.54
REINFORCE - Episode: 300 Average Reward: 57.18
```




```
========== COMPARISON RESULT ==========
              Algorithm  Final 50 Episode Avg  Best Episode Reward
Vanilla Policy Gradient                 12.98                 48.0
              REINFORCE                 57.18                188.0

Result: REINFORCE performed better based on the final 50-episode average reward.
```

A moving-average learning curve is plotted to compare the performance of both algorithms.

The graph contains:

- **X-axis:** Episode
- **Y-axis:** Average Reward
- **Blue curve:** Vanilla Policy Gradient
- **Orange curve:** REINFORCE

The graph shows that REINFORCE achieves substantially higher rewards toward the later stages of training.

RESULTS AND OBSERVATIONS

The experiment was conducted for 300 episodes.

For Vanilla Policy Gradient, the average rewards reported at different stages were:

- Episode 50: **17.86**
- Episode 100: **13.94**
- Episode 150: **15.96**
- Episode 200: **13.70**
- Episode 250: **12.54**
- Episode 300: **12.98**

For REINFORCE, the average rewards were:

- Episode 50: **22.08**
- Episode 100: **24.80**
- Episode 150: **30.16**
- Episode 200: **29.78**
- Episode 250: **41.54**
- Episode 300: **57.18**

The REINFORCE learning curve shows a strong improvement during the later episodes, while the Vanilla Policy Gradient reward remains comparatively low.

COMPARISON RESULT

Vanilla Policy Gradient

- Final 50-episode average reward: **12.98**
- Best episode reward: **48.0**

REINFORCE

- Final 50-episode average reward: **57.18**

- Best episode reward: 188.0

Therefore, REINFORCE performed better based on the final 50-episode average reward.

The improvement in final average reward is approximately:

57.18 − 12.98 = 44.20

Thus, REINFORCE achieved a 44.20 higher final 50-episode average reward than Vanilla Policy Gradient in this experiment.

EXPECTED RESULT

The program is expected to train both policy-gradient algorithms on the CartPole-v1 environment and provide a quantitative comparison of their learning performance.

The algorithm achieving the higher final average reward and better learning curve is considered to have performed better for the selected experiment.

CONCLUSION

Vanilla Policy Gradient and REINFORCE were successfully implemented and compared using the CartPole-v1 reinforcement learning environment.

Based on the obtained results, REINFORCE performed better than Vanilla Policy Gradient in this particular experiment. REINFORCE achieved a final 50-episode average reward of 57.18, compared with 12.98 for Vanilla Policy Gradient. It also achieved a higher best episode reward of 188.0, compared with 48.0 for Vanilla Policy Gradient.

The experiment demonstrates how policy-gradient methods can be evaluated using cumulative rewards and learning curves. The results also show that algorithmic differences in the treatment of returns can significantly affect learning behavior and final performance.

Overall, REINFORCE provided better learning performance for the CartPole-v1 environment under the selected training settings.